

Internationalization & Configuration

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/StuderApplication.java

Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java

Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java

Backend/STUDER/src/main/resources/application-prod.properties

Backend/STUDER/src/main/resources/application.properties

Backend/STUDER/src/main/resources/messages.properties

Backend/STUDER/src/main/resources/messages_es.properties

Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts

This section details the backend configuration and internationalization (i18n) infrastructure of STUDER. The system uses a localized message resolution strategy based on HTTP headers and maintains distinct environment profiles for development and production.

Internationalization (i18n)

The backend implements i18n to provide localized error messages and system notifications. This is primarily handled by the `I18nConfig` class and standard Spring `MessageSource` resource bundles.

Locale Resolution

The system uses an `AcceptHeaderLocaleResolver` to determine the client's preferred language. It extracts the locale from the `Accept-Language` header sent by the frontend

Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java 15 If no header is present, it defaults to English Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java 16

Message Bundles

Localized strings are stored in `.properties` files under `src/main/resources`. The application is configured to use `messages` as the base name with UTF-8 encoding

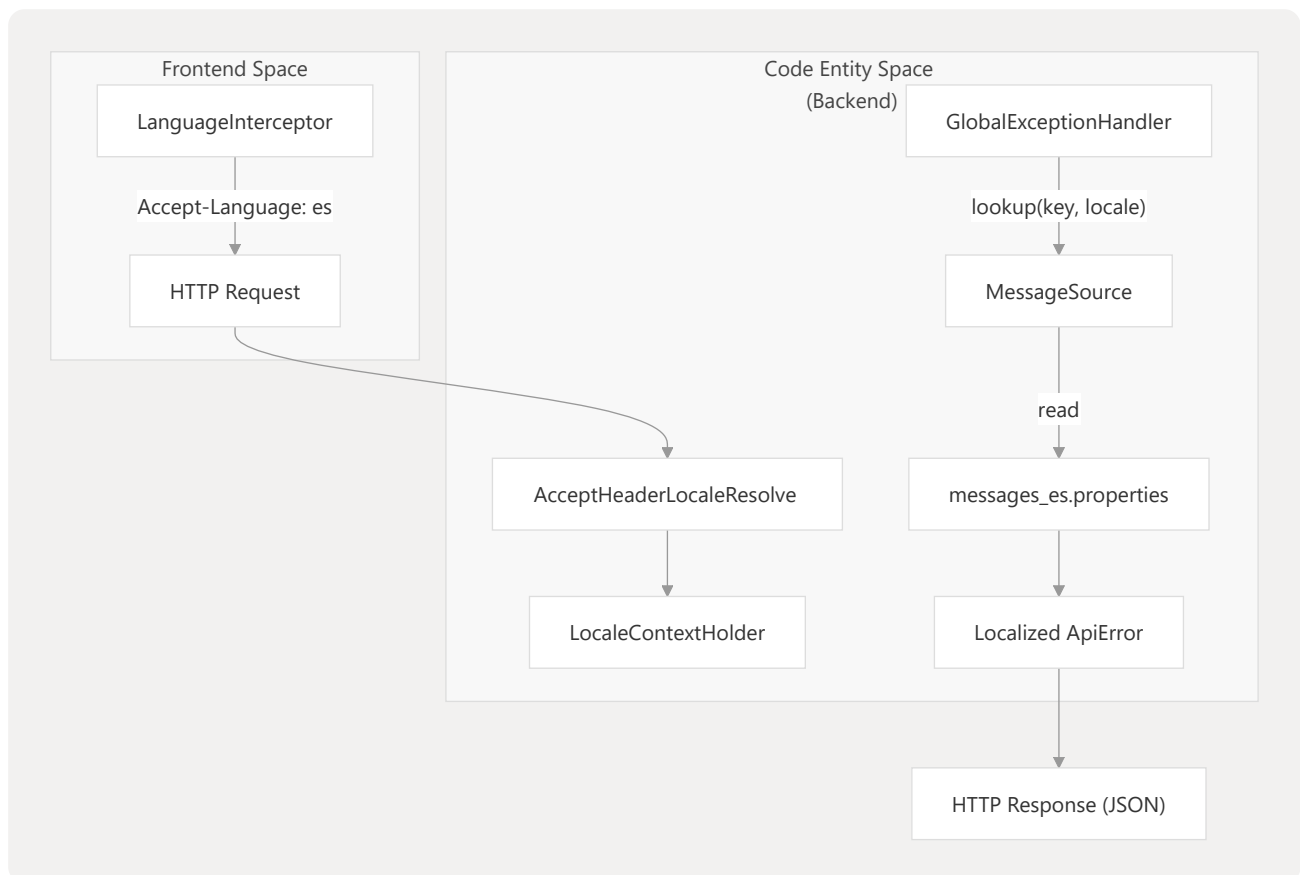
Backend/STUDER/src/main/resources/application.properties 3-4

- **messages.properties** : Contains default English translations for user validation, auth, and domain entities Backend/STUDER/src/main/resources/messages.properties 1-64
- **messages_es.properties** : Contains Spanish translations for the same keys Backend/STUDER/src/main/resources/messages_es.properties 1-74

Data Flow: Locale to Response

The following diagram illustrates how a request from the frontend results in a localized error message.

I18n Request Flow



Sources: Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java 13-18

Backend/STUDER/src/main/resources/application.properties 3-4

Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts 34-61

Application Configuration

STUDER utilizes Spring Profiles to manage different environments. The primary configuration is defined in `application.properties`, which activates the `dev` profile by default

Backend/STUDER/src/main/resources/application.properties

7

Configuration Profiles

Feature	<code>application.properties</code> (Base)	<code>application-prod.properties</code> (Production)
Server Port	8081 Backend/STUDER/src/main/resources/application.properties22	8081 Backend/STUDER/src/main/resources/application-prod.properties20
Media URL	<code>http://studer-media:8000</code> Backend/STUDER/src/main/resources/application.properties25	<code>http://studer-media:8000</code> Backend/STUDER/src/main/resources/application-prod.properties22
CORS Origins	Localhost 3000, 4200, 1583 Backend/STUDER/src/main/resources/application.properties20	Localhost 3000, 4200, 1583 Backend/STUDER/src/main/resources/application-prod.properties19
Database	N/A (Inherited/Dev)	PostgreSQL with Environment Variables Backend/STUDER/src/main/resources/application-prod.properties1-3
Auth	JWT Secret, Token Expiry Backend/STUDER/src/main/resources/application.properties6-14	JWT Secret via Env Var Backend/STUDER/src/main/resources/application-prod.properties18

System Startup

Upon startup, the `StuderApplication` class initializes the context

Backend/STUDER/src/main/java/facu/studer/StuderApplication.java

13-15

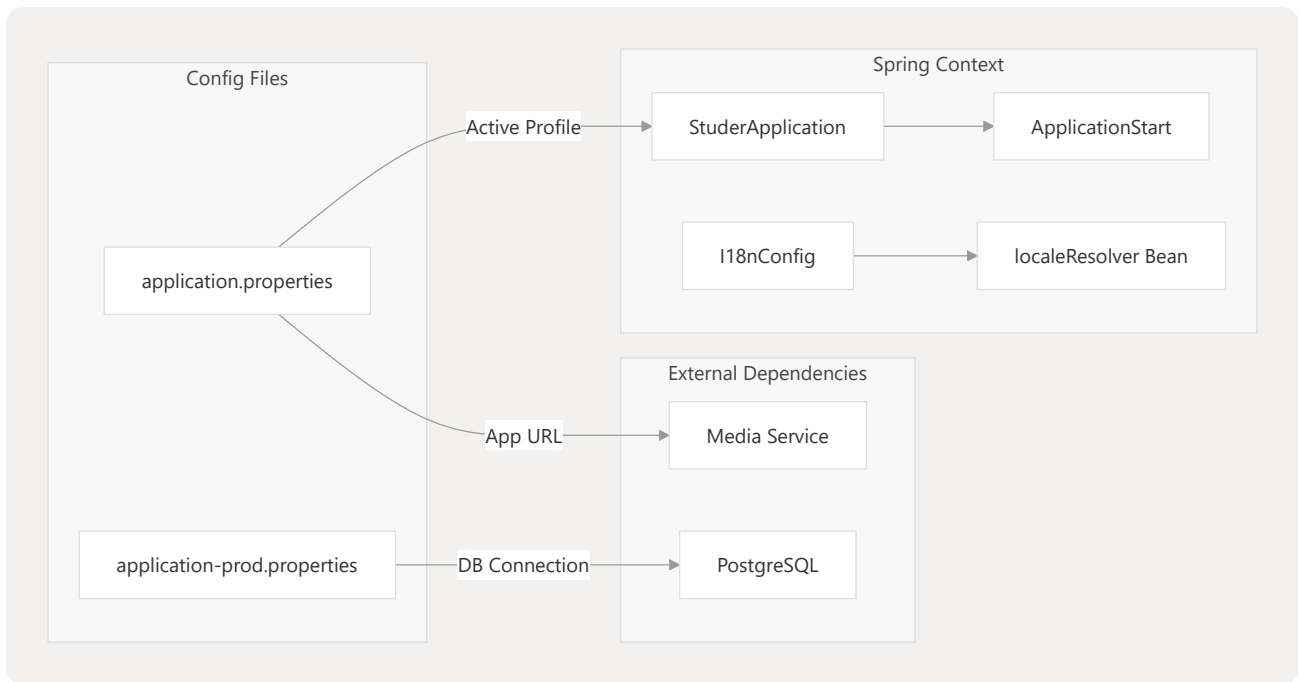
The `ApplicationStart`

component listens for the `ContextRefreshedEvent` to print a stylized ASCII banner to the logs, indicating the system is ready

Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java

52-152

Configuration and Initialization Map



Sources: Backend/STUDER/src/main/java/facu/studer/StuderApplication.java 6-15

Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java 15-17

Backend/STUDER/src/main/resources/application.properties 1-26

Case Conversion Interceptor

While not strictly i18n, the system handles data consistency across the boundary via the **caseConverterInterceptor**. This ensures that the backend receives **snake_case** (standard for Python/Postgres) and the frontend receives **camelCase** (standard for TypeScript).

- **Outgoing Requests:** Converts JSON body to **snake_case**
Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts 48-52
- **Incoming Responses:** Converts JSON body to **camelCase**
Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts 55-57
- **Exclusions:** It ignores requests for assets or **FormData** (used for image uploads)
Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts 39-45

Sources: Frontend/STUDER/src/app/core/http/case-converter.interceptor.ts 8-61